

Тема: ПР 22 Применение библиотек Pygame

Цель: совершенствование навыков составления программ с библиотекой Pygame

Задачи:

- ✓ повторить структуру алгоритмов решения задач с библиотекой Pygame;
- ✓ повторить синтаксис операторов работы с графикой;
- ✓ приобрести навыки составления программ с графикой.

Теоретический материал

Pygame задает особые правила построения кода. Эти правила не являются строгими. Однако в большинстве случаев, чтобы игра благополучно запустилась, в программе должна быть соблюдена определенная последовательность вызова ключевых команд.

Эти команды (импорт модуля, вызовы функций, цикл) создают своего рода скелет, или каркас, программного кода. Выполнив его, вы получите "пустую" игру. Далее на этот скелет "подвешивается мясо", т. е. объявляются объекты и программируется логика игры.

Первое, что нужно сделать, это импортировать модуль pygame. После этого можно вывести на экран главное графическое окно игры с помощью функции `set_mode()` модуля `display`, входящего в состав библиотеки `pygame`:

```
import pygame
pygame.display.set_mode((600, 400))
```

Если выполнить этот код, то появится окно размером 600x400 пикселей и сразу закроется (в Linux, в Windows может зависнуть).

Функция `set_mode()` принимает три аргумента – размер в виде кортежа из двух целых чисел, флаги и глубину цвета. Их можно не указывать. В этом случае окно займет весь экран, цветовая глубина будет соответствовать системной. Обычно указывают только первый аргумент – размер окна.

Флаги предназначены для переключения на аппаратное ускорение, полноэкранный режим, отключения рамки окна и др. Например, команда `pygame.display.set_mode((640, 560), pygame.RESIZABLE)` делает окно изменяемым в размерах.

Выражение вида `pygame.RESIZABLE` (вместо `RESIZABLE` может быть любое другое слово большими буквами) обозначает обращение к той или иной константе, определенной в модуле `pygame`. Часто можно встретить код, в котором перед константами не пишется имя модуля (вместо, например, `pygame.QUIT` пишут просто `QUIT`). В этом случае в начале программы надо импортировать не только `pygame`, но и содержимое модуля `locals` через `from ... import`:

```
import pygame
from pygame.locals import *
```

Однако в данном курсе мы оставим длинное обращение к встроенным константам, чтобы на этапе обучения не путать определенные в модуле и свои собственные, которые нам также придется создавать.

Функция `set_mode()` возвращает объект типа `Surface` (поверхность). В программе может быть множество объектов данного класса, но тот, что возвращает `set_mode()` особенный. Его называют `display surface`, что можно перевести как экранная (дисплейная) поверхность. Она главная.

В конечном итоге все отображается на ней с помощью функции `pygame.display.update()` или родственной `pygame.display.flip()`, и именно эту поверхность мы видим на экране монитора. Нам пока нечего отображать, мы не создавали никаких объектов. Поэтому было показано черное окно.

Функции `update()` и `flip()` модуля `display` обновляют содержимое окна игры. Это значит, что каждому пикселю заново устанавливается цвет. Представьте, что на зеленом фоне движется красный круг. За один кадр круг смещается на 5 пикселей. От кадра к кадру картинка целого окна изменяется незначительно, но в памяти окно будет перерисовываться полностью. Если частота составляет 60 кадров в секунду ($FPS=60$), то за секунду в памяти компьютера произойдет 60 обновлений множества значений, соответствующих экранным пикселям, что дает по большей части бессмысленную нагрузку на вычислительные мощности.

Если функции `update()` не передавать аргументы, то будут обновляться значения всей поверхности окна. Однако можно передать более мелкую прямоугольную область или список таковых. В этом случае обновляться будут только они.

Функция `flip()` решает проблему иным способом. Она дает выигрыш, если в `set_mod()` были переданы определенные флаги (аппаратное ускорение + полноэкранный режим – `pygame.HWSERFACE|pygame.FULLSCREEN`, двойная буферизация – `pygame.DOUBLEBUFF`, использование OpenGL – `pygame.OPENGL`). Возможно, все флаги можно комбинировать вместе (через `|`). При этом, согласно документации, аппаратное ускорение работает только в полноэкранном режиме.

Сначала отключается `pygame`, потом происходит выход из программы. Такой вариант вероятно следует считать наиболее безопасным завершением. Команда `pygame.quit()` не обязательна. Если завершается программа, то отключится и `pygame`.

Другой вариант – не допустить следующей итерации цикла. Для этого потребуется дополнительная переменная:

```
import pygame as pg

pg.display.set_mode((600, 400))

play = True
while play:
    for i in pg.event.get():
        if i.type == pg.QUIT:
            play = False
```

В этом случае завершится текущая итерация цикла, но новая уже не начнется. Если в основной ветке ниже по течению нет другого кода, программа завершит свою работу.

Нередко код основной ветки программы помещают в функцию, например, `main()`. Она выполняется, если файл запускается как скрипт, а не импортируется как модуль. В этом случае для завершения программы проще использовать оператор `return`, который осуществляет выход из функции.

```
import pygame as pg

def main():
    pg.display.set_mode((600, 400))

    while True:
        for i in pg.event.get():
            if i.type == pg.QUIT:
                return

if __name__ == "__main__":
    main()
```

Теперь зададимся вопросом, с какой скоростью крутится цикл `while`? С большой, зависящей от мощности компьютера. Но в данном случае такая скорость не есть необходимость, она даже вредна, так как бессмысленно расходует ресурсы. Человек дает команды и воспринимает изменения куда медленнее.

Для обновления экрана в динамической игре часто используют 60 кадров в секунду, а в статической, типа пазла, достаточно будет 30-ти. Из этого следует, что циклу незачем работать быстрее.

Поэтому в главном цикле следует выполнять задержку. Делают это либо вызовом функции `delay()` модуля `time` библиотеки `pygame`, либо создают объект часов и устанавливают ему частоту кадров. Первый способ проще, второй – более профессиональный.

```
...
while 1:
    for i in pg.event.get():
        if i.type == pg.QUIT:
            sys.exit()
    pg.time.delay(20)
```

Функция `delay()` принимает количество миллисекунд ($1000 \text{ мс} = 1 \text{ с}$). Если передано значение 20, то за секунду экран обновится 50 раз. Другими словами, частота составит 50 кадров в секунду.

```
import pygame as pg
import sys

pg.display.set_mode((600, 400))

clock = pg.time.Clock()

while 1:
    for i in pg.event.get():
        if i.type == pg.QUIT:
            sys.exit()
    clock.tick(60)
```

Методу `tick()` класса `Clock` передается непосредственно желаемое количество кадров в секунду. Задержку он вычисляет сам. То есть если внутри цикла указано `tick(60)` – это не значит, что задержка будет 60 миллисекунд или произойдет 60 обновлений экрана за одну итерацию цикла. Это значит, что на каждой итерации цикла секунда делится на 60 и уже на вычисленную величину выполняется задержка.

Практические задания.

Создайте каркас своей игры (собираение каких-либо объектов), подберите нужные изображения и фон

Контрольные вопросы:

- 1) Какие использовали модули для работы вашей программы?
- 2) Какие функции используются для работы вашей программы?

Домашнее задание добавьте на выбор 3 функции для своей программы